Definizione sintassi con esempi

1 Overview del programma

Il programma del linguaggio **pike** è costituito da una sequenza (da 0 a più) di istruzioni. Queste possono essere definizioni di funzioni o singoli statement eseguiti in ordine.

1.1 Tipi

La grammatica supporta due tipi primitivi:

- `num`: Rappresenta valori numerici (decimali).
- `str`: Rappresenta stringhe di testo racchiuse tra virgolette doppie (ad esempio `"testo"`).

1.2 Commenti

I commenti iniziano con il carattere `#` e continuano fino alla fine della riga.

2 Funzioni

2.1 Dichiarazione

Le funzioni vengono definite con la keyword `defunc`.

- Devono avere un nome e una lista di argomenti tra parentesi tonde.
- Ogni argomento deve avere un tipo esplicito (`nome: tipo`).
- Il tipo di ritorno è opzionale ed è indicato dopo il simbolo `%`. Se è inserito, bisogna inserire uno statement di return nel corpo della funzione.
- Il corpo della funzione è racchiuso tra parentesi graffe `{ ... }`.

Sono supportate le funzioni senza argomenti.

```
# Funzione con ritorno
defunc somma(a: num, b: num) % num {
    # Corpo...
}

# Procedura senza ritorno esplicito
defunc printer(s: str) {
```

```
    # Corpo...
}
```

3 Statement

Gli statement sono di due tipi:

- *Semplici* (assegnazioni, return, I/O, espressioni): devono terminare con un punto e virgola `;`.
- *Blocchi di controllo flusso* (`if`, `for`): non richiedono il punto e virgola finale.

3.1 Assegnazioni

L'assegnazione utilizza l'operatore `:=`. Assegna il risultato di un'espressione al nome di una variabile.

```
x := 10 + 5;
messaggio := "Ciao";
```

3.2 Dichiarazione variabili

Non esiste una keyword esplicita per la dichiarazione (come `var` o `let`). Le variabili vengono dichiarate implicitamente al momento della prima assegnazione.

```
# La variabile tmp viene creata qui
tmp := 0 + 0;
```

3.3 Output

Esistono due comandi per l'output a schermo:

1. `!`: Stampa l'espressione senza andare a capo.
2. `!!`: Stampa l'espressione seguita da un ritorno a capo (newline).

```
!("Stampa senza a capo");
!!("Stampa con a capo");
!!(x);
```

3.4 Return

Per restituire un valore da una funzione si utilizza la keyword `ret`.

```
ret result;
ret 0;
```

3.5 If-Else

La struttura condizionale utilizza `if` seguito da una condizione tra parentesi tonde e un blocco di codice tra graffe. È supportato il blocco opzionale `else`.

```
if (a > 1) {
    result := 10;
} else {
    result := 0;
}
```

3.6 Loops

Il ciclo supportato è il `for` in stile C. Richiede tre sezioni separate da punto e virgola all'interno delle parentesi tonde: inizializzazione, condizione e incremento/aggiornamento.

```
for (i := 0; i < 10; i := i + 1) {
    !!(i);
}
```

3.7 Espressioni

Ci sono tre tipi di espressioni.

3.7.1 Operazioni aritmetiche e di confronto

Le espressioni supportano la precedenza degli operatori standard.

- **Confronto:** `<` (minore), `>` (maggiore).
- **Additivi:** `+` (più), `-` (meno).
- **Moltiplicativi:** `*` (per), `/` (diviso).

```
# Il risultato è 14 = 2 + (3 * 4)
valore := 2 + 3 * 4;
check := i < result;
```

3.7.2 Chiamate a funzione/procedura

Le funzioni vengono chiamate utilizzando il loro nome seguito dagli argomenti tra parentesi tonde. Una chiamata a funzione può essere usata come statement autonomo (se non restituisce nulla o se il risultato viene ignorato) o come parte di un'espressione.

```
r := somma(2, 3);  # Usata in un'assegnazione
printer(r);        # Usata come statement
```